

Investigation of RBLapSum **through_rank_kappa** instability

Research notebook

Started 2026-09-16. Investigation initiated after the France kappa campaign.

Contents

1	The question	2
2	Plain-language glossary	2
3	What we know going in	4
4	Candidate mechanisms	4
5	Evidence plan	6
6	Evidence phase 1: outcomes, onset, and early indicators	7
7	Evidence phase 2: measured surrogate forces	9
8	Theory v3: burst-driven escape	11
9	Falsification wave 1	12
10	Adaptive temperature controller: initial design	14
11	Wave 2: pre-registered predictions	16
12	Additional rank-resolved force analysis	16
13	Wave 1 results	18
14	Wave 2.0: the first controller was mis-calibrated	19
15	A forward-only geometric pressure measure	20
16	Wave 2.1 mid-flight results	21
17	Wave 2.1 final results	22
18	Interpretation of the null-test failure	22

19 Controller revision: protective-only temperature adaptation	23
20 Per-layer temperatures	24
21 Current interpretation	24
22 Remaining uncertainties	25
23 Summary	25

1 The question

The `through_rank_kappa` gradient mode resolved the severe instability previously observed with `through_rank` (see `docs/figures/rb_divergence_*.png`). At $K = 32$, $T = 1.0$, it is currently the best-performing method in the campaign: validation CE reaches 1.491 at $J = 480$, compared with the previous LapSum best of 1.545.

However, the campaign revealed three classes of instability that were not initially understood:

observation	detail
$K = 64$, $J = 448$, $T = 1.0$ diverges	The run is healthy at 2k steps (probe-batch CE 1.81), begins to destabilize around 3–4k steps (CE 5.9 at 4k), and is fully collapsed by 10k. The otherwise identical $T = 2.0$ run remains healthy and finishes at 1.472. At $K = 64$, $T = 1.0$, the $J = 64$ and $J = 192$ runs are also stable.
$T = 0.5$ at $K = 32$ is usu-ally unstable	$J = 32, 96, 480$ collapse. $J = 224$ survives, with a modest degradation: 1.5125 compared with 1.4940 at $T = 1$.
$T = 0.1$ at $K = 32$ is al-ways unstable	All four tested J values diverge, with final CE around 6 or worse.

The goals of this investigation are:

1. identify the mechanism responsible for the instability;
2. determine whether the K -axis failure ($K : 32 \rightarrow 64$ at fixed $T = 1$) and the temperature-axis failure ($T : 1 \rightarrow 0.5 \rightarrow 0.1$ at fixed $K = 32$) arise from the same mechanism;
3. derive a principled rule for choosing T , or modify the algorithm so that the unstable regime is avoided automatically.

2 Plain-language glossary

- **Score.** For AbsTopK,

$$s_i = |z_i|.$$

The score measures the magnitude of feature i at a given token. The gate keeps the K largest features active and additionally tracks the next J features as backward-pass candidates.

- **Boundary.**

$$b = \max(0.1, s_{(K+1)}).$$

Here $s_{(K+1)}$ is the score of the first inactive feature. Thus b is the local threshold separating the active support from the inactive candidate region.

- **Kernel.**

$$\kappa_T(s - b) = \frac{e^{-|s-b|/T}}{2T}.$$

This is an exponential kernel centered at the boundary. The surrogate support gradient is weighted by κ_T , so features near the boundary receive the strongest support-learning signal.

Its characteristic width is T , while its maximum value is

$$\kappa_{\max} = \frac{1}{2T}.$$

Reducing T therefore makes the kernel narrower and increases its peak magnitude.

- **Raw surrogate term.**

$$a_i = u_i z_i \kappa_i, \quad u_i = \frac{\partial L}{\partial y_i}.$$

Here u_i is the ordinary task-loss gradient arriving at the bottleneck output. The quantity a_i is the independent local contribution that would be obtained before applying the collective correction.

- **Kappa correction.**

$$g_i = a_i - q_i \sum_j a_j, \quad q_i = \frac{\kappa_i}{\sum_j \kappa_j}.$$

The correction removes the collective score-gradient component while distributing the compensating term among candidates in proportion to their kernel weights.

This distinguishes `through_rank_kappa` from `detach`, where no collective correction is applied.

- **Feature preference signal.** Define

$$w_i = -u_i z_i.$$

With this sign convention, $w_i > 0$ means that the task loss locally prefers feature i to contribute more strongly.

A gradient-descent step in score space has the schematic form

$$\Delta s_i \propto \kappa_i (w_i - \langle w \rangle_q),$$

where

$$\langle w \rangle_q = \sum_i q_i w_i.$$

Thus a near-boundary feature moves upward when it is preferred more strongly than the kernel-weighted local average, and downward when it is preferred less.

- **Local spacing.** Let

$$\delta(b) = \frac{s_{(K-3)} - s_{(K+5)}}{8}.$$

This estimates the typical spacing between neighboring feature ranks near the boundary. Small δ means that many features occupy a narrow score interval around the support threshold.

- **Effective window population.**

$$n_{\text{eff}} = \frac{(\sum_i \kappa_i)^2}{\sum_i \kappa_i^2}.$$

This is an effective-number-of-elements measure for the kernel weights. It estimates how many candidates meaningfully participate in the local boundary correction.

3 What we know going in

- Previous analysis of `through_rank`, where the entire boundary correction is assigned to the single $(K + 1)$ -st feature, identified the following failure mode: the boundary feature can receive an anomalously large update; activation scale then grows through depth along weakly constrained scale directions; the boundary eventually moves far outside the kernel region; the support gradient vanishes; and optimization becomes trapped at a very large activation scale.
- `detach`, which applies no collective correction, was stable for 20k steps at $K = 32$, $T = 1.0$. Thus a collective correction is not required for stability in that particular regime.
- The $K = 64$ campaign shares the same architecture and training setup as the $K = 32$ runs except for K and J : 8×768 , MD/decouple, `abs_topk`, `residual_out`, $b_0 = 0.1$, and the same learning-rate schedule.
- `uk_rout_rblapsum_kappa_k64_j448_md_abs` and `...k32_j480...` both track 512 features in total. The main structural difference is therefore the location of the support boundary: rank 65 versus rank 33. This suggests that the local score geometry at the boundary, rather than the total number of candidates alone, may be important.

4 Candidate mechanisms

The following hypotheses were formulated before the detailed measurements.

H1 — Degeneration of the kappa correction

As $T \rightarrow 0$, the kernel weights may become concentrated on only one or a few features near the boundary. In the extreme case,

$$n_{\text{eff}} \rightarrow 1,$$

and the distributed correction approaches the original `through_rank` behavior, where the collective correction is effectively carried by a single feature.

Prediction: instability should correlate with small n_{eff} .

A difficulty for this hypothesis is the K -dependence. At fixed T , the rank-65 boundary lies in a denser part of the score distribution than rank 33. One might therefore initially expect a larger, not smaller, n_{eff} at $K = 64$.

H2 — Excessive surrogate-force magnitude

The maximum per-candidate kernel weight is

$$\kappa_{\text{max}} = \frac{1}{2T}.$$

If the local score density is

$$\rho(b) \approx \frac{1}{\delta(b)},$$

then for the exponential kernel,

$$\sum_i \kappa_i^2 \approx \frac{\rho(b)}{4T}.$$

Thus reducing T increases both the peak local gradient and the total squared surrogate weight in the boundary region.

This hypothesis qualitatively explains both observed axes:

$$T \downarrow \Rightarrow \text{stronger kernel,}$$

and

$$K \uparrow \Rightarrow \text{potentially larger local density } \rho(b).$$

The main quantitative question is whether the measured change in local density is large enough to account for the $K = 32 \rightarrow 64$ transition.

H3 — Boundary churn and cumulative scale drift

Define a dimensionless comparison between the characteristic surrogate-induced score displacement and the local spacing between ranks:

$$\chi = \frac{\text{characteristic boundary-region score displacement}}{\delta(b)}.$$

The intended interpretation is that when the surrogate moves near-boundary features by an amount comparable to their rank spacing, the identities of near-boundary features can change rapidly from step to step.

A possible cumulative mechanism is then:

1. the correction is zero-sum within the current candidate set;
2. a feature pushed upward can leave the boundary region and retain its increased score;
3. a feature pushed downward can leave the candidate set or move to a region with negligible kernel weight;
4. consequently, exact cancellation at one instant does not necessarily imply cancellation over a sequence of changing candidate sets.

This can create a small systematic bias in score dynamics even when each individual correction is zero-sum.

Using the rough local scale

$$\text{kick} \sim \frac{\eta|u|b}{2T},$$

one obtains

$$\chi \propto \frac{\eta|u|b}{2T\delta(b)}.$$

This predicts instability when the boundary region is simultaneously:

- sharply weighted by the kernel (T small),
- located at large score scale b ,
- and densely populated (δ small).

It also predicts weak direct dependence on J at fixed local geometry.

H4 — Switching between rank and floor regimes

Because

$$b = \max(b_0, s_{(K+1)}),$$

it is possible in principle that deeper boundaries, such as $K = 64$, cross the floor $b_0 = 0.1$. This would switch the backward rule between a rank-dependent and a fixed-boundary regime.

This hypothesis can be tested directly using `rb_cap_active_frac`.

Relation between H2 and H3

H2 and H3 are closely related. Both propose that the kernel becomes too strong relative to the local score geometry.

H3 is more specific because it introduces the local rank spacing as the comparison scale and proposes a dynamical mechanism by which repeated near-boundary updates can accumulate.

H1 concerns a different limit: the distributed correction itself ceases to be distributed once the effective kernel population becomes very small.

5 Evidence plan

1. Run outcomes and onset timing.

Use the France TensorBoard scalars (`analysis/kappa_stability.py tb`) to determine when each run destabilizes and whether `rb_support_grad_norm`, `rb_boundary`, or `rb_cap_active_frac` changes first.

2. Score geometry.

Use the `lens_1` score ladders (`... ladder`) to measure

$$\delta, \quad b, \quad \sum_i \kappa_i, \quad n_{\text{eff}}$$

for each layer and checkpoint.

Measure these both at each run’s actual boundary and counterfactually at ranks 33 and 65 in the same activation tensors. This isolates the geometric effect of moving the boundary deeper into the same score distribution.

3. Training-time surrogate forces.

Use probe captures (`... probe`) to reconstruct

$$a, \quad q, \quad g_s$$

from captured (z, u) , and verify the reconstruction against the captured gradient with respect to z .

Measure:

- local surrogate-gradient magnitudes;
- $\text{Cov}_q(d, w)$, which characterizes stretching or compression of the boundary band;
- support churn between probes;
- cumulative asymmetries in how upward- and downward-moving candidates leave the kernel region.

4. Falsification runs.

Use the available France 8×5090 machine to test predictions that distinguish the candidate mechanisms.

6 Evidence phase 1: outcomes, onset, and early indicators

The following results were extracted with `analysis/kappa_stability.py tb` from the France TensorBoard logs.

The reported onset is the first step whose training loss exceeds the best loss so far by 1 nat.

K	J	T	onset	final CE	K	J	T	onset	final CE
32	32	0.1	640	5.98	32	32	1.0	—	1.548
32	96	0.1	640	9.44	32	96	1.0	—	1.525
32	224	0.1	520	5.98	32	224	1.0	—	1.494
32	480	0.1	580	5.98	32	480	1.0	—	1.491
32	32	0.5	3760	6.08	32–64, all		2.0	—	1.47–1.56
32	96	0.5	6480	5.53	64	64	1.0	—	1.504
32	224	0.5	—	1.512	64	192	1.0	—	1.489
32	480	0.5	7760	5.77	64	448	1.0	3940	14.04

Table 1: Outcomes of the 22 campaign runs. “—” indicates survival through 20k training steps.

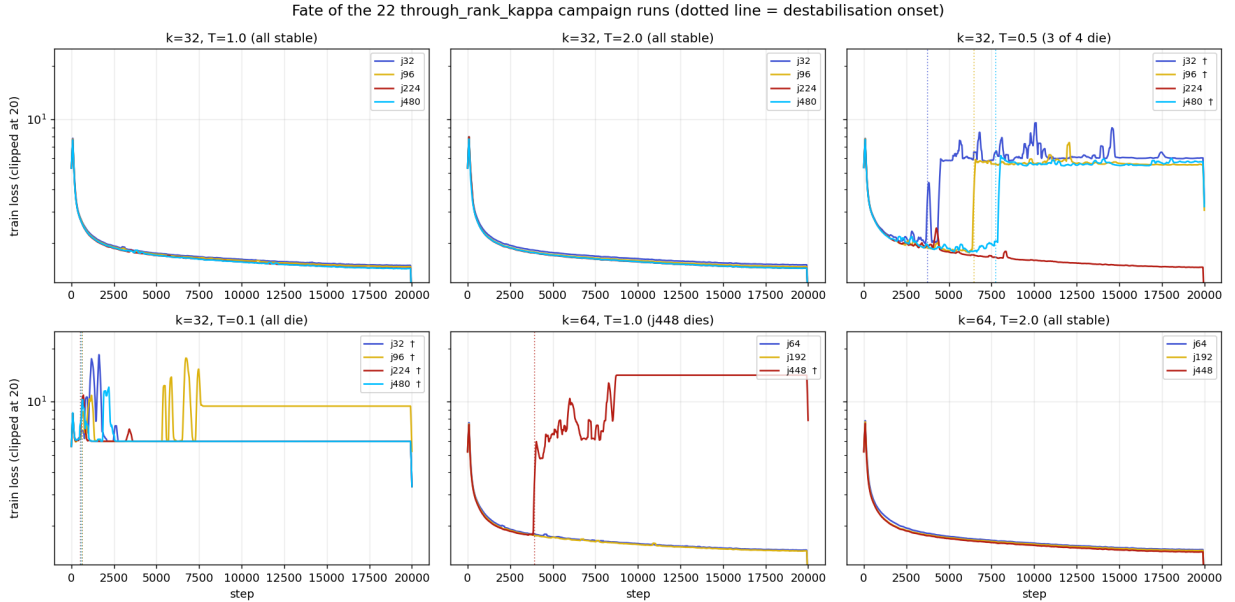


Figure 1: Training-loss outcomes for the 22 `through_rank_kappa` campaign runs. Dotted lines indicate the measured destabilization onset.

Several observations from the TensorBoard signals are particularly useful.

- `rb_cap_active_frac` remains 1.0 throughout all runs, including during collapse.

Therefore the b_0 floor is not active in the observed failures, which rules out H4 for this campaign.

- `rb_common_mode` remains around 10^{-6} .

Thus the kappa correction continues to remove the collective score-gradient component accurately even while the model destabilizes. The failure is not caused by a simple numerical loss of the zero-sum property.

- The boundary score b begins to increase substantially before the task loss shows visible deterioration.

For the unstable $T = 0.5$ runs, b can remain between 25 and 500 times its baseline value for hundreds or thousands of steps before the measured onset.

For $K = 64$, $J = 448$, $T = 1$, the last 1000 steps before onset show approximately a 22% increase in b and a 4.4-fold increase in gradient norm.

- `rb_support_grad_norm` does not show a smooth pre-onset divergence.

The surrogate support gradient therefore does not simply explode first and directly drive the task loss upward. Instead, the available evidence is more consistent with the surrogate slowly moving the model into an unstable part of parameter/activation space, after which the ordinary task gradients become large.

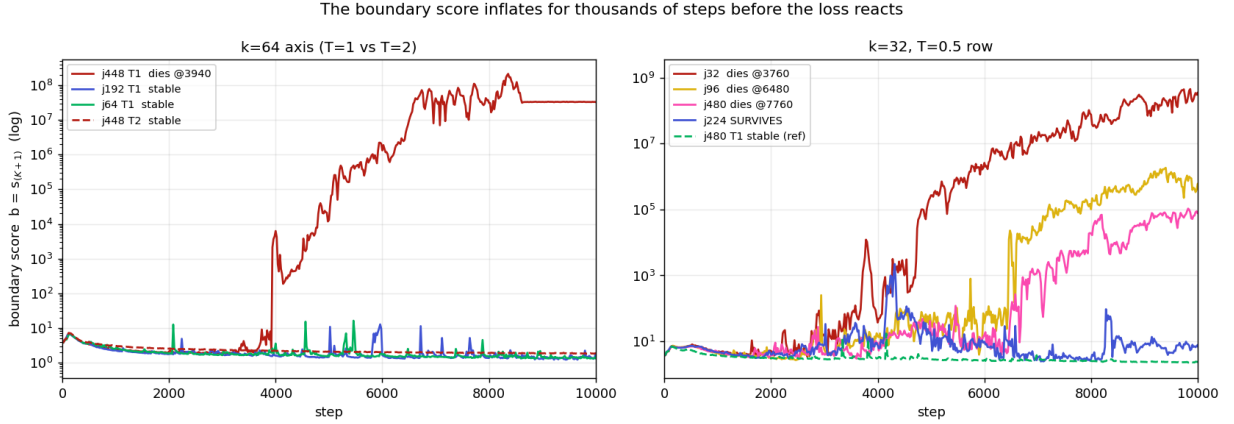


Figure 2: Boundary-score inflation can precede visible loss degradation by hundreds or thousands of training steps. Left: the $K = 64$ comparison. Right: the $K = 32$, $T = 0.5$ runs.

A further observation changes the picture from a simple monotonic drift to a metastable process. Marginal runs show repeated transient excursions of the boundary score, often followed by recovery. In some cases the excursions reach factors of 10^2 – 10^3 relative to the local baseline.

For example, the surviving $K = 32$, $J = 224$, $T = 0.5$ run undergoes two approximately 10^3 -fold excursions near step 4300 and recovers from both.

In unstable runs, the final failure appears to correspond to an excursion from which the system does not recover, followed by persistent scale growth.

For clarity, we refer to these transient excursions as *inflation bursts*.

The table below counts excursions for which b exceeds three times its rolling median, normalized per 1000 pre-onset steps.

configuration	burst rate	outcome
$K = 32$, $T = 2$ and $K = 64$, $T = 2$	0.00	all stable; maximum excursion ≤ 1.9
$K = 32$, $T = 1$	0.05–0.21	all stable
$K = 64$, $T = 1$	$J64 : 0.37$, $J192 : 0.42$, $J448 : 0.97$	$J448$ diverges
$K = 32$, $T = 0.5$	$J224 : \mathbf{0.52}$, $J480 : 1.7$, $J32 : 3.1$, $J96 : 3.2$	$J224$ is the only survivor

Within these rows, burst rate is associated with outcome: the unstable $K = 64$ run has the highest burst rate in its row, while the surviving $T = 0.5$ run has the lowest rate by a factor of roughly 3–6.

This does not by itself establish a causal role for burst rate, but it provides a useful empirical marker for proximity to the unstable regime.

7 Evidence phase 2: measured surrogate forces

The command `analysis/kappa_stability.py probe` reconstructs the surrogate gradient per candidate from probe captures using

$$u = \frac{\partial L}{\partial \tilde{z}}$$

together with the signed z .

The reconstruction agrees with the captured

$$\frac{\partial L}{\partial z}$$

to approximately zero relative error at the reported precision, so the quantities below correspond to the actual training-time backward rule.

The $T = 0.1$ collapse

Consider $K = 32$, $J = 96$, $T = 0.1$, layer 4.

Here “kick” denotes the mean $|g_s|$ among candidates in the kernel region, and n_{eff} measures the effective number of candidates sharing that region.

step	probe CE	kick	n_{eff}	$s_{(K+1)}$	top- K mean
0	11.30	$7.2 \cdot 10^{-3}$	22	3.8	4.3
100	7.18	$5.7 \cdot 10^{-5}$	15	4.9	5.7
150	6.20	$1.9 \cdot 10^{-4}$	3	11.9	15.6
250	6.10	$6.1 \cdot 10^{-4}$	2	36	47
350	5.84	$1.7 \cdot 10^{-6}$	1	50	73
500	6.06	~ 0	1	664	1297
1000	17.2	~ 0	1	21941	49688

The observed sequence is:

1. The initial kernel is very sharp:

$$\kappa_{\text{max}} = \frac{1}{2T} = 5.$$

2. Scores begin to increase and move away from the original boundary region.
3. The effective kernel population falls:

$$n_{\text{eff}} : 22 \rightarrow 15 \rightarrow 3 \rightarrow 2 \rightarrow 1.$$

4. Once $n_{\text{eff}} \approx 1$, the distributed kappa correction effectively approaches the single-boundary-feature limit of `through_rank`.
5. Activation scale then increases rapidly.
6. Eventually the candidates move so far from the kernel that the support gradient itself becomes nearly zero.
7. At that stage the model remains at an activation scale several orders of magnitude above the initial regime, while the task loss is badly degraded.

The support dynamics change correspondingly. At the beginning of the run, approximately 96% of the top- K support is replaced over 10 steps, compared with roughly 30% in healthy runs. Later, after the activation scale has become very large, the support freezes and the overlap approaches 1.

A kick-to-spacing diagnostic

Define

$$\chi = \frac{\overline{|g_s|}_{\text{window}}}{\delta(b)}.$$

This quantity compares the surrogate gradient scale near the boundary with the local score spacing. Strictly speaking, the expression above is in gradient units rather than literal per-optimizer-step displacement units. A true displacement criterion would also depend on the optimizer and effective step size. Nevertheless, when computed under the same training setup, this quantity is useful for comparing runs.

Measured over steps 300–1000:

configuration	χ (layer mean)	outcome
$K = 32, T = 2$	$2.7 \cdot 10^{-5}$	stable
$K = 64, T = 2$	$5.4 \cdot 10^{-5}$	stable
$K = 32, T = 1$	$8.2 \cdot 10^{-5}$	stable
$K = 64, T = 1$	$1.5\text{--}1.7 \cdot 10^{-4}$, similar across all three J	marginal; 1 of 3 diverges
$K = 32, T = 0.5$	$1.9 \cdot 10^{-4}$	marginal; 3 of 4 diverge
$K = 32, T = 0.1$	$\sim 10^{-2}$ at initialization	diverges within several hundred steps

Within the measured campaign, χ is primarily a property of the (K, T) row and varies little with J .

The empirical ordering is:

$$\text{stable regime} \lesssim 10^{-4},$$

$$\text{marginal regime} \sim 1.5\text{--}2 \times 10^{-4},$$

while $T = 0.1$ begins roughly two orders of magnitude above these values.

The measured scaling is consistent with

$$\chi \propto \frac{|u| b}{2T \delta(b)}.$$

The boundary at rank 65 lies in a substantially denser score region than the boundary at rank 33. Measured inside the same activation tensors,

$$\frac{\delta_{33}}{\delta_{65}} \approx 3.3\text{--}4.6.$$

The rank-65 boundary also has approximately 30% smaller b . Combining these effects gives

$$\frac{\chi(K = 64, T = 1)}{\chi(K = 32, T = 1)} \approx 2.2,$$

which is close to

$$\frac{\chi(K = 32, T = 0.5)}{\chi(K = 32, T = 1)}.$$

Thus, with respect to this local pressure measure,

$$K = 64, T = 1$$

is approximately comparable to

$$K = 32, T \approx 0.5.$$

The measured ratio of safe temperatures,

$$\frac{T_c(64)}{T_c(32)} \approx 2.2,$$

is consistent with the observation that $T = 2$ is stable at $K = 64$.

This supports the hypothesis that the K -axis and T -axis failures arise from the same underlying interaction between kernel width and local score geometry.

Two additional observations

- In healthy steady states,

$$n_{\text{eff}} \approx 46\text{--}446,$$

far from the $n_{\text{eff}} \approx 1$ regime.

Thus H1 does not explain the ordinary operating state. Instead, degeneration of the correction appears to occur only during severe inflation episodes and may act as a terminal amplification mechanism once the window has already collapsed.

- A small cumulative asymmetry is measurable even in healthy runs.

Candidates receiving upward surrogate updates fall approximately 0.04 score units less over 10 steps than candidates receiving downward surrogate updates, against a mean regression of approximately -0.5 .

Thus the surrogate contributes only a few percent bias to the background score dynamics in the healthy regime. This is consistent with a gradual modification of the score distribution rather than direct domination of the optimizer.

8 Theory v3: burst-driven escape

The evidence above suggests the following working model.

1. The kernel applies persistent pressure to the boundary region. The relevant strength is controlled by the kernel scale relative to the local score geometry, summarized empirically by χ .
2. The network can partially accommodate this pressure by changing the overall score scale. As the scale increases, fewer candidates remain strongly weighted by the fixed-width kernel. The ordinary task loss provides an opposing force.
3. At low enough pressure, these effects remain in a stable regime. This is consistent with the absence of bursts at $T = 2$ and the low burst rate at $K = 32, T = 1$.
4. In the marginal regime, transient score-inflation excursions become more frequent.

5. If an excursion reduces the effective kernel population to

$$n_{\text{eff}} = O(1),$$

the distributed kappa correction approaches its single-feature `through_rank` limit.

Previous experiments indicate that this limit strongly favors further activation-scale growth.

6. The excursion can then become self-sustaining: the score scale increases, the kernel population shrinks further, and eventually the support gradient itself becomes negligible because almost all features lie far outside the kernel.

The model is then left in a high-activation-scale state with poor task loss.

7. At $T = 0.1$, this sequence occurs reproducibly within the first few hundred steps.

At intermediate pressures, failure is stochastic: individual runs may undergo several recoverable bursts before one becomes unrecoverable.

9 Falsification wave 1

These runs use 6k-step observations within the same 20k-step training schedules.

The following predictions were recorded before the runs.

run	question	prediction under theory v3
<i>K64, J448, T1</i> detach	Is the kappa correction itself responsible for the failure?	If the raw kernel force is the main source of the scale pressure, detach should also be unstable at this operating point.
<i>K64, J448, T1</i> project	Does uniform common-mode removal behave differently from kappa-weighted removal?	Expected behavior between detach and kappa.
<i>K64, J448, T1</i> , seed 1338	Is the failure at step 3940 deterministic?	If failure is an escape process, a different seed should have a different onset and may survive substantially longer.
<i>K64, J448, T = 1.5</i>	Bracket the pressure threshold.	With estimated $\chi \approx 1.0\text{--}1.1 \times 10^{-4}$, the run should lie near the stable edge.
<i>K64, J320, T1</i>	Test the role of J within the marginal <i>K64, T1</i> row.	Since χ is similar across the row, outcome should depend mostly on stochastic burst occurrence rather than a deterministic J threshold.
<i>K32, J480, T = 0.75</i>	Bracket the threshold along the <i>K32</i> temperature axis.	Expected to be near the stable side of the marginal region.
<i>K32, J96, T0.5</i> , seed 1338	Test reproducibility of the unstable $T = 0.5$ regime.	Expected to diverge, but not necessarily at the same step as the original run.
5k probe of <i>K64, J448, T1</i>	Capture gradients during a possible failure.	If a collapse occurs, n_{eff} should fall strongly during the final inflation episode.

Early observations at approximately step 1100 already showed:

- the **detach** run had CE 9.0 and infinite gradient norm;

- the `project` run had CE 7.1 and infinite gradient norm;
- the corresponding kappa-corrected runs remained healthy at the same stage.

This is important because `detach` had previously been stable for 20k steps at $K = 32, T = 1$.

Thus the distributed kappa correction is not simply the source of instability. At the higher-pressure $K = 64, T = 1$ operating point it provides substantially more protection than either `detach` or the uniform projection.

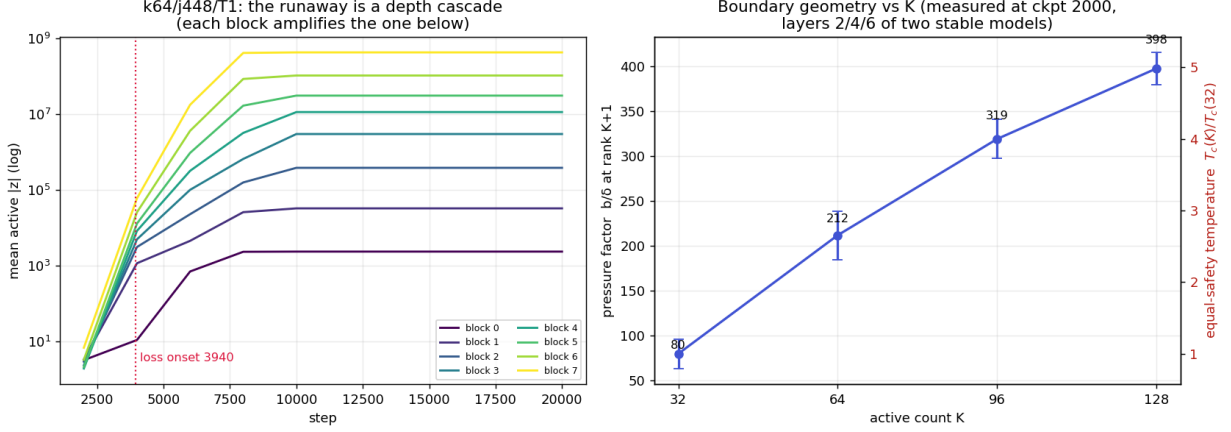


Figure 3: Left: after the onset of severe instability, activation-scale growth propagates through depth. With `residual_out`, each block can amplify the scale passed from the preceding block, eventually reaching bf16 saturation. Right: the geometric pressure factor b/δ grows approximately with K over the measured range.

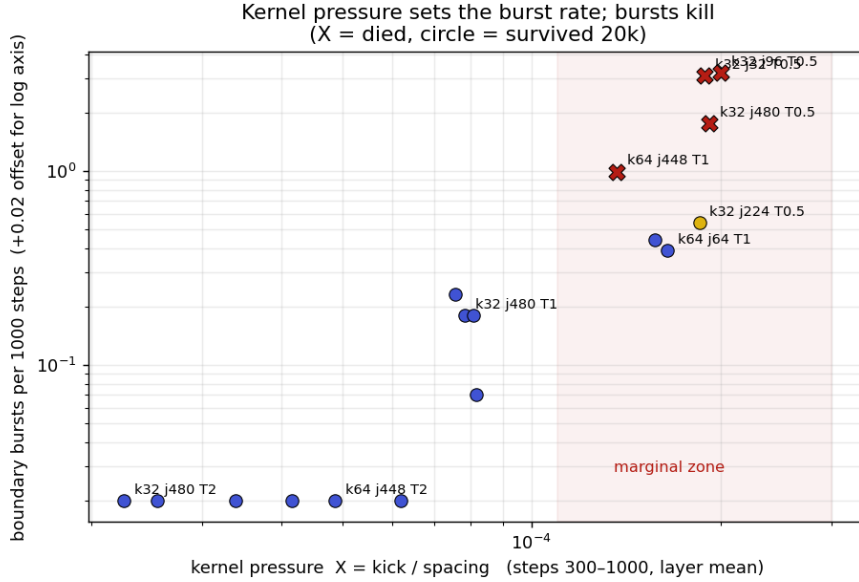


Figure 4: Observed relation between kernel-pressure diagnostics and instability. Crosses indicate runs that diverged; circles indicate runs that survived 20k steps.

The $T = 0.1$ example also clarifies the ordering of events.

The effective kernel population reaches approximately one by step 350 while the task CE is still relatively normal. The boundary score is already increasing rapidly at this stage. The measured support kick then falls toward zero as the kernel loses its effective population around step 450. The task loss does not visibly deteriorate until roughly step 800.

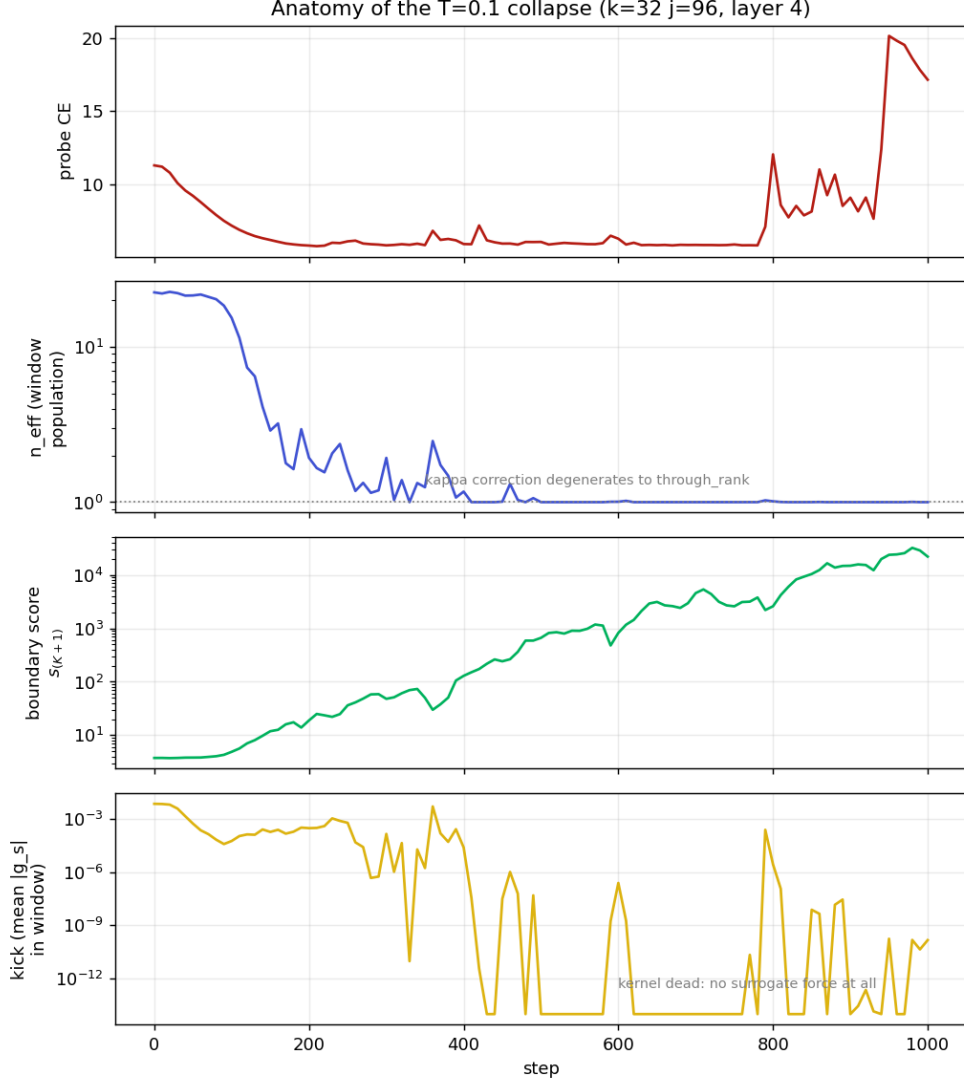


Figure 5: Detailed evolution of the $K = 32$, $J = 96$, $T = 0.1$ collapse in layer 4.

Thus the task loss is a delayed indicator of the underlying score-scale instability.

10 Adaptive temperature controller: initial design

The working theory identifies two relevant conditions:

1. sustained excessive kernel pressure;
2. a sufficiently severe inflation episode that reduces the effective boundary population to $O(1)$.

Both can in principle be measured by the gate itself. This motivates feedback control of T rather than a fixed temperature schedule or a lookup table indexed by K .

The initial implementation was

```
rblapsum_temperature_mode: servo.
```

It is implemented in `src/wsparse/bottleneck/{gate,rblapsum}.py`, with a separate scalar T per layer stored in persistent buffers.

The first design used two mechanisms.

Population floor

If fewer than

$$\text{rblapsum_window_floor} = 16$$

candidates occupy the kernel window on a batch, increase T by 10% on that step.

The purpose is to prevent the effective boundary population from collapsing toward the single-feature regime.

During an inflation episode, the boundary moves away from nearby candidates and the kernel population falls. Increasing T broadens the window and attempts to restore a distributed correction.

Because the increase compounds as 1.1^n , the controller can respond by a large factor within tens of steps.

Kick-to-spacing trim

Otherwise, adjust T by at most 2% per step to target

$$\frac{\text{EMA}(\text{kick})}{\text{EMA}(\delta)} = \text{rblapsum_chi_target} = 8 \times 10^{-5}.$$

This value was chosen from the measured $K = 32, T = 1$ regime, which combined good validation performance with stable training.

The intended behavior was approximately:

$$T \approx 1 \quad \text{at } K = 32,$$

$$T \gtrsim 2 \quad \text{at } K = 64,$$

with larger values chosen automatically for larger K .

Why two simpler rules were rejected

- Setting T proportional to the local score spread is scale-invariant, which is attractive during activation inflation.

However, the rank-65 boundary lies in a denser region than rank 33. A spread-following rule would therefore decrease T at larger K , whereas the experiments show that larger K requires a larger T .

- Using a fixed $T = 2$ is sufficient for the tested $K \leq 64$ cases, but the measured geometric factor

$$\frac{b}{\delta}$$

increases with K :

$$80, 212, 319, 398 \quad \text{for } K = 32, 64, 96, 128.$$

Therefore a single fixed temperature is unlikely to remain appropriate as K increases. It may also unnecessarily smooth the boundary at small K , where $T = 1$ gives better quality.

11 Wave 2: pre-registered predictions

The score geometry was measured at checkpoint 2000 from stable campaign models before launching the following runs.

run		prediction
kstab_k96_j416_t1 ($T = 1$)	(fixed	Estimated pressure is roughly $1.5\times$ that of the unstable $K64, J448, T1$ configuration. Prediction: unstable, with onset earlier than step 3940; broad expected range approximately 1.5–6k.
kstab_k96_j416_servo		Prediction: stable through 20k, with settled T above the $K = 64$ servo temperature.
kstab_k64_j448_servo seed 1338	and	Prediction: both survive beyond step 3940. Final CE expected to be close to or better than the fixed- $T = 2$ result of 1.472.
kstab_k32_j480_t05_servo ($T_0 = 0.5$)		Prediction: the controller increases T out of the marginal region and prevents the failure seen with fixed $T = 0.5$. Expected final CE around 1.49.
kstab_k32_j96_t01_servo ($T_0 = 0.1$)		Prediction: the population guard increases T by approximately an order of magnitude quickly enough to prevent the early fixed- $T = 0.1$ collapse.
kstab_k32_j480_servo ($T_0 = 1.0$)		Null test: the controller should remain near $T = 1$ and retain approximately the 1.491 quality of the fixed- $T = 1$ reference run.

12 Additional rank-resolved force analysis

This section was added after the initial wave-2 predictions.

For $K = 64, T = 1$, layer 4, the reconstructed rank-averaged surrogate force over steps 300–1000 has a characteristic sign change across the support boundary.

The marginal active features, roughly ranks 33–64, have

$$w < 0,$$

meaning that the task loss locally prefers these features to contribute less.

The near inactive tail, roughly ranks 65–128, is instead pushed upward.

The top-ranked active features behave differently: the top 16 have the strongest positive preference signal, approximately

$$w \approx 9 \times 10^{-7},$$

but their kernel weights are small, approximately

$$\kappa \approx 0.05,$$

because they lie far from the support boundary.

Thus, in its healthy regime, the support surrogate primarily acts on marginal features: it pushes weak marginal active features downward and promising near-boundary inactive features upward.

This is qualitatively the behavior expected from a support-swap optimizer.

Two additional observations are useful.

- The uncorrected collective term

$$\sum_i a_i$$

is approximately 4–5 times larger at $J = 64$ than at $J = 448$, because the shorter candidate tail provides less cancellation.

Nevertheless, $J = 64$ is stable while $J = 448$ is the unstable run in the $K = 64, T = 1$ row.

This provides further evidence that the uncorrected common mode is not the main driver of the observed instability; the kappa correction removes it effectively.

- At $T = 1$, the kernel still has substantial weight throughout the full candidate set even for $J = 448$:

$$\kappa \approx 0.18$$

at rank 512.

The effective populations are approximately

$$n_{\text{eff}} = 102, 206, 373$$

for

$$J = 64, 192, 448.$$

Within the $K = 64, T = 1$ row, burst rate increases with this population:

$$0.37, 0.42, 0.97$$

bursts per 1000 steps.

However, the $T = 0.5$ row does not follow the same ordering. For example, $J = 32$ has the smallest candidate window but one of the highest burst rates.

Therefore candidate population alone does not explain which member of a marginal row fails.

13 Wave 1 results

run	predicted	observed	assessment
<i>K64, J448, T1 detach</i>	unstable if raw kernel pressure is sufficient	diverged within the first 100 steps; final frozen CE 9.06	consistent; failure is substantially faster than the kappa run's step-3940 failure
<i>K64, J448, T1 project</i>	intermediate between detach and kappa	diverged within 100 steps; frozen CE 7.09	uniform common-mode projection provides little protection in this regime
<i>K64, J448, T1, seed 1338</i>	different failure time under a stochastic-escape picture	survived 6k with CE 1.687 and burst rate 0.39/1k	literal prediction of another failure was not confirmed; result supports the weaker claim that this configuration defines a failure rate rather than a deterministic failure time
<i>K64, J448, T1.5</i>	= near stable edge	survived 6k with zero detected bursts; maximum excursion $1.5\times$	stable and somewhat calmer than expected
<i>K64, J320, T1</i>	marginal, outcome not determined solely by J	survived; burst rate 0.39/1k	consistent
<i>K32, J480, T0.75</i>	= near stable edge	survived with CE 1.730 and burst rate 0.19/1k	consistent
<i>K32, J96, T0.5, seed 1338</i>	unstable with seed-dependent onset	diverged at step 3680, versus 6480 for seed 1; one excursion reached approximately $10^6\times$	consistent
5k probe through <i>K64, J448, T1</i>	capture $n_{\text{eff}} \rightarrow O(1)$ if a fatal burst occurs	this replica did not fail, but revealed two additional effects described below	partial test

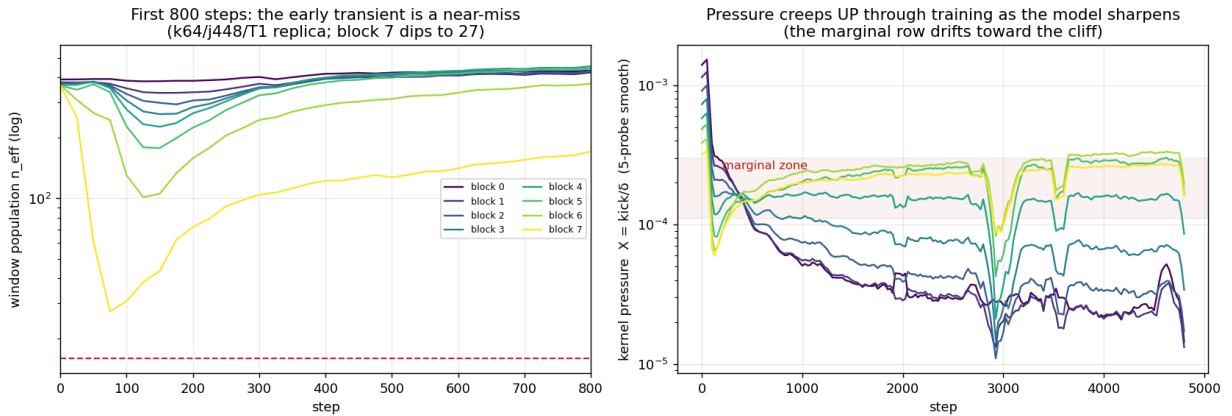


Figure 6: Gradient-resolved replica of *K64, J448, T1*. Left: early reduction of the effective kernel population. Right: gradual increase of the pressure diagnostic during training.

The gradient-resolved replica suggests two refinements.

- The early training transient is a high-risk period.

Within approximately the first 100 steps, the effective kernel population in deep blocks drops strongly; for example, block 7 changes from approximately 363 to 27 while χ initially reaches around 10^{-3} .

The methods that fail almost immediately— $T = 0.1$, detach, and project—all fail during this phase.

Runs that remain stable first recover from this transient.

- **Pressure can increase gradually later in training.**

After the early recovery, χ rises again over time. In block 7 it increases from approximately

$$1.2 \times 10^{-4}$$

to

$$2.7 \times 10^{-4}$$

by step 4800.

The main reason is that the score distribution sharpens: δ decreases faster than the surrogate kick decreases.

Thus late failures need not arise solely because the model waits for a rare event at a fixed operating point. The operating point itself can become progressively less stable.

- **The relevant geometry differs substantially across layers.**

Blocks 5–7 occupy the marginal regime while blocks 0–4 relax toward lower pressure.

This motivates controlling temperature separately by layer rather than with a single model-wide scalar.

After wave 1, the main revision is therefore:

A marginal configuration should be viewed as having a nonzero, potentially time-dependent failure rate, rather than as being guaranteed to diverge at a particular step.

14 Wave 2.0: the first controller was mis-calibrated

Approximately 800 steps into the first adaptive-temperature experiments, telemetry showed that the kick-based trim was not calibrated correctly.

Healthy runs were reducing T aggressively; for example, the $K = 64$ controller moved from approximately

$$T = 0.97$$

to

$$T = 0.45,$$

eventually operating near the population-floor intervention.

The source of the mismatch is that the measured

$$|g_s|$$

contains the task loss’s per-token normalization.

The probe captures used to calibrate the target were computed with approximately 256-token batches, while live training steps contain roughly 50k tokens.

Thus an absolute threshold expressed in task-gradient units changes with batch normalization and does not transfer between the probe and training settings.

This invalidates the use of the original gradient-unit χ as a portable controller target.

The population floor itself remained useful: the $T_0 = 0.1$ rescue had already increased T to approximately 0.5 while maintaining healthy loss, whereas the corresponding fixed- $T = 0.1$ run had already failed.

15 A forward-only geometric pressure measure

To remove the dependence on gradient normalization, define

$$\chi_{\text{geo}} = \frac{b}{2T\delta}.$$

This quantity depends only on forward score geometry and is dimensionless.

At checkpoint 2000, using the maximum value across layers:

- every measured run with

$$\chi_{\text{geo}} \lesssim 55$$

survived;

- every observed failure had

$$\chi_{\text{geo}} \gtrsim 58;$$

- the $K = 64, T = 1$ row sits around

$$\chi_{\text{geo}} \approx 114,$$

where outcome is strongly seed-dependent.

Within the measured runs, the additional u -dependent factor in the gradient-based χ provides relatively little discriminating information beyond this score geometry.

The controller was therefore redesigned to target

$$\chi_{\text{geo}} = \frac{\text{EMA}(b)}{2T \text{EMA}(\delta)}.$$

The initial target was set to 45, slightly below the measured $K = 32, T = 1$ operating point of approximately 51.

Using measured

$$\frac{b}{\delta} \approx 80, 212, 319$$

for

$$K = 32, 64, 96,$$

the pre-registered expected temperatures were approximately:

$$T \approx 0.9\text{--}1.2 \quad (K = 32),$$

$$T \approx 2\text{--}2.6 \quad (K = 64),$$

$$T \approx 3\text{--}4 \quad (K = 96).$$

The purpose of this wave was to test whether a controller using only local score geometry could recover the empirically safe temperature scale.

16 Wave 2.1 mid-flight results

At approximately step 2400:

run	T_0	rb_temp @2.4k	rb_chi	window	loss	pre-registered T
k32_j480_servo	1.0	0.947	46.4	129	1.919	0.9–1.2 ✓
k32_j480_t05_servo	0.5	0.946	37.2	143	1.942	rescue toward same operating point ✓
k32_j96_t01_servo	0.1	1.039	44.8	103	1.905	rescue from early-collapse regime ✓
k64_j448_servo	1.0	2.268	45.0	450	1.884	2–2.6 ✓
k96_j416_servo	1.0	3.001	45.5	483	1.908	3–4 ✓

The three $K = 32$ runs started from very different temperatures:

$$1.0, \quad 0.5, \quad 0.1.$$

By step 2400, all had converged to approximately

$$0.95\text{--}1.04.$$

Similarly, the $K = 64$ and $K = 96$ runs settled around

$$2.27 \quad \text{and} \quad 3.00,$$

respectively.

These values are approximately proportional to the previously measured

$$b/\delta = 80, \ 212, \ 319.$$

Thus, at this stage of training, the geometric controller recovered the expected K -dependent temperature scale without an explicit K -lookup table.

In parallel, `kstab_k96_j416_t1` with fixed $T = 1$ diverged by approximately step 3800, within the pre-registered 1.5–6k interval.

This is an important out-of-sample test because $K = 96$ had not previously been trained successfully in this setup, and the instability prediction was made from score geometry measured before the run.

17 Wave 2.1 final results

run	outcome	final CE	reference
kstab_k64_j448_servo	stable	1.4691	fixed $T = 2$: 1.4724; fixed $T = 1$: diverged at 3940
kstab_k64_j448_servo_s2	stable	1.4608	best $K = 64$ result in the project
kstab_k96_j416_servo	stable	1.4734	fixed $T = 1$: diverged at 1640
kstab_k32_j480_t05_servo ($T_0 = 0.5$)	stable; one recoverable burst near 8.9k	1.5075	fixed $T = 0.5$: diverged at 7760
kstab_k32_j96_t01_servo ($T_0 = 0.1$)	stable; one recoverable burst near 5.1k; guard raised T to 2.2 during the event	1.5261	fixed $T = 0.1$: diverged within 640 steps; fixed $T = 1$, same J : 1.525
kstab_k32_j480_servo ($T_0 = 1$, null test)	destabilized at approximately 4140	3.945	fixed $T = 1$: stable, 1.491

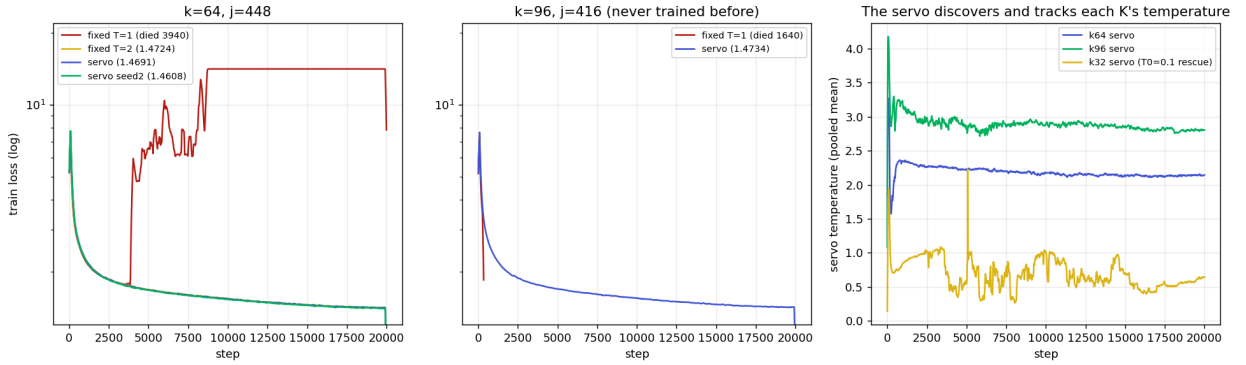


Figure 7: Adaptive versus fixed temperature. Left: $K = 64$, $J = 448$. Middle: $K = 96$, $J = 416$. Right: per-layer temperatures selected by the controller.

18 Interpretation of the null-test failure

The failed $K = 32$, $J = 480$, $T_0 = 1$ null test revealed a flaw in the symmetric temperature controller.

The controller attempted to maintain

$$\chi_{\text{geo}} = 45$$

by moving T both upward and downward.

Between approximately steps 4k and 8k, it reduced T to the range

$$0.49\text{--}0.70,$$

which overlaps the empirically marginal $T \approx 0.5\text{--}0.75$ regime.

The reason is that the safe operating value of χ_{geo} is not constant during training.

The stable fixed- $T = 1$, $K = 32$ reference run starts around

$$\chi_{\text{geo}} \approx 39$$

and gradually falls toward approximately 30.

Therefore forcing the model back toward 45 requires progressively decreasing T , even though the model is already in a stable regime.

The controller was consequently sharpening the kernel beyond a temperature known to be safe. In the failed null-test run, the boundary score increased for several thousand steps before a late excursion reached

$$b \approx 2500$$

and became unrecoverable.

The two $K = 32$ runs that began from initially unsafe temperatures happened to survive, while the nominally safe null test failed after the controller moved it into the marginal region.

This is consistent with the earlier observation that marginal operating points have a stochastic failure rate rather than a deterministic outcome.

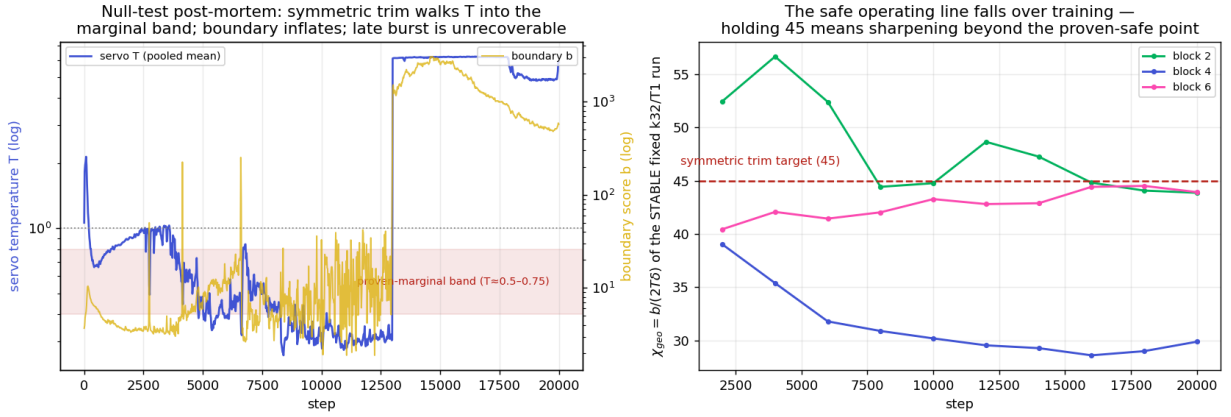


Figure 8: Null-test analysis. Left: the symmetric controller reduces T into the marginal regime before the final boundary excursion. Right: in the stable fixed- $T = 1$ $K = 32$ run, χ_{geo} decreases naturally during training, so forcing it upward to a fixed target unnecessarily sharpens the kernel.

19 Controller revision: protective-only temperature adaptation

The controller has therefore been changed from a symmetric regulator to a protective mechanism. The revised rule imposes

$$T \geq T_{\text{configured}},$$

where $T_{\text{configured}}$ is the baseline temperature supplied by the experiment.

The controller may increase T when the measured geometry becomes unsafe, but it does not decrease T below the baseline in an attempt to improve sharpness or quality.

This asymmetry is motivated directly by the experiments:

- increasing T above the baseline prevents instability in the high-pressure $K = 64$ and $K = 96$ regimes;
- decreasing T below a known-safe baseline moves the model toward the documented $T = 0.5$ – 0.75 marginal region;
- the fixed baseline already provides the desired quality in the low-pressure regime, so there is limited justification for taking this additional stability risk.

Wave 2.2 repeats the null test with this lower temperature bound using two random seeds, together with a repeated $K = 96$ run.

20 Per-layer temperatures

The final checkpoints of the successful controller runs show substantial layer-to-layer variation.

For $K = 64$, blocks 0 through 7 settle approximately at

$$2.74, 2.48, 2.11, 1.83, 1.82, 1.99, 1.97, 2.21.$$

The profile is roughly U-shaped across depth.

For $K = 96$, the same qualitative pattern appears at a larger overall scale, approximately

$$3.52, \dots, 2.06, \dots, 3.06.$$

The temperature varies by about a factor of 1.5 within a single model.

This supports the earlier conclusion that the relevant score geometry is layer-dependent and that a single global temperature is unlikely to be optimal across all bottlenecks.

21 Current interpretation

The evidence currently supports the following picture.

1. The instability is not explained by loss of the zero-sum property. The kappa correction continues to remove the collective score-gradient mode during unstable runs.
2. The instability is strongly associated with the relationship between kernel width and local score geometry.

Both decreasing T and increasing K can move the system into a regime where the kernel is too sharp relative to the density of scores near the support boundary.

3. The $K = 32 \rightarrow 64$ instability and the $T = 1 \rightarrow 0.5$ instability are consistent with the same mechanism.

The measured local score density quantitatively accounts for the approximate factor-of-two change in the required temperature.

4. Instability typically develops through changes in activation scale before large changes in task loss become visible.

Boundary-score inflation is therefore a more sensitive early indicator than CE.

5. Marginal configurations exhibit transient inflation bursts. Whether a particular trajectory eventually diverges is stochastic, and the failure rate itself can increase over training as the score distribution sharpens.

6. A particularly dangerous state occurs when the effective kernel population falls toward

$$n_{\text{eff}} = O(1).$$

At that point the distributed correction approaches the single-boundary-feature `through_rank` limit.

7. The purely geometric quantity

$$\chi_{\text{geo}} = \frac{b}{2T\delta}$$

captures the relevant variation across the tested values of K , T , and training time substantially better than an absolute gradient-unit threshold, because it is independent of loss normalization.

8. Adaptive temperature control is effective when used protectively: increase T when local geometry becomes unsafe, but do not reduce it below a known-safe configured baseline.

22 Remaining uncertainties

Several aspects remain unresolved.

- The precise mechanism that determines which run fails within a marginal (K, T) row is not fully explained.

For $K = 64, T = 1$, burst rate increases with effective candidate population, but the $T = 0.5$ row does not show the same ordering with J .

- The proposed cumulative asymmetry in near-boundary feature motion is measurable but small in healthy runs.

More direct evidence is still needed to determine exactly how this small bias couples to global activation-scale growth.

- The role of the downstream architecture, particularly `residual_out`, deserves separate analysis. Once scale growth becomes severe, the current architecture allows it to propagate and amplify through depth, but this does not by itself identify the initial cause of the instability.
- The empirical thresholds for χ_{geo} are based on the present architecture, optimizer, and training setup. The quantity is dimensionless and transfers better than the earlier gradient-unit criterion, but the numerical safe range should not yet be treated as universal.
- The protective controller has been validated on $K = 32, 64, 96$ in the current campaign, but additional K , architecture, and seed sweeps are required to determine how broadly the rule generalizes.

23 Summary

The original question was why `through_rank_kappa` can be stable and high-performing at $K = 32, T = 1$, yet fail when either K is increased or T is decreased.

The measurements support a common explanation.

The relevant control parameter is not K or T separately, but the sharpness of the kernel relative to the local score spacing near the $K/(K + 1)$ boundary.

Increasing K moves the boundary into a denser part of the score distribution. Reducing T sharpens the kernel. Both therefore increase the pressure exerted on the near-boundary region.

At sufficiently high pressure, score-scale excursions become frequent. A severe excursion can reduce the effective kernel population toward one, at which point the distributed correction loses its main stabilizing advantage and approaches the previously unstable single-boundary-feature limit.

A forward-only diagnostic,

$$\chi_{\text{geo}} = \frac{b}{2T\delta},$$

captures the observed K - and T -dependence and predicts the temperature scale required at previously untested K .

An adaptive, per-layer controller can use this geometry to increase T when necessary. The controller should be protective rather than symmetric: it may broaden the kernel when the local geometry becomes unsafe, but should not sharpen the kernel below a known-safe configured temperature merely to maintain a fixed target value of χ_{geo} .